Course Code: ITA1439 **Course Name: Ethical Hacking**

Slot: A

CO2 – ASSIGNMENT_2

Nature of Assignment: Individual **Duration: 1 Week** **Total Marks: 50**

Student Name: A. Govardhan Reddy **Reg. No: 192421359**

CO2: Execute foot printing and social-engineering techniques such as DNS analysis, competitive intelligence gathering and attack simulations to collect information for identifying potential security risks.

Assignment Title: Evaluation and Engineering of Foot printing, Reconnaissance and Social-Engineering-Awareness Workflows in an Authorized Assessment

Scenario - Assessment Design Philosophy

The assignment is designed for high-performing cybersecurity students and emphasizes the evaluation of reconnaissance and information-gathering choices rather than the mechanical use of tools. Students are expected to formulate an information-gathering strategy, construct difficult test conditions, run footprinting against authorized targets, collect evidence, identify limitations, compare alternatives and defend engineering decisions. All work is performed only against authorized or lab-owned targets, and no attack is launched against live systems or real individuals.

The assessment follows: Understand → Predict → Design → Collect → Analyse → Evaluate → Defend → Validate.

MARK DISTRIBUTION

Question	Assessed Area	Marks
Q1	Footprinting strategy design and evaluation	8
Q2	DNS footprinting and analysis workflow	7
Q3	Competitive-intelligence and awareness utility	7
Q4	Reconnaissance data validation and recovery	6
Q5	Hierarchical organization footprinting	7
Q6	Footprinting approach comparison	7
Q7	Integrated diagnose–redesign–validate challenge	8
TOTAL		**50**

0. Scope, authorization and method

Authorization statement. Every target in this report is a **lab-owned, synthetic** name under the reserved `.example` top-level domain (RFC 2606), answered by a BIND 9 instance on loopback addresses (127.0.0.0/8) or an in-memory synthetic document corpus. No packet was sent to a real organisation or domain, no real person is named, and no social-engineering attack was carried out against anyone. Real tools were used where the assignment asks for them: `whois`, `dig`, `nslookup` and BIND itself. Anything that could not be real (WHOIS registry, a search engine, certificate-transparency data, public documents) is a **lab simulation**, and the report says so wherever it matters.

Method followed (Understand → Predict → Design → Collect → Analyse → Evaluate → Defend → Validate). For every question, predictions were written to a file and hashed **before** the collection script was run. The hashes are listed below, so the prediction-versus-observation tables are not written after the fact. Where an observation disagreed with a prediction, the disagreement is stated, not hidden.

Prediction file	Used in	SHA-256 (first 16 hex)	Written (UTC)
q1_q2_predictions.json	Q1, Q2	c91e70ce4bc77d7e	2026-11-02 00:59:26
q1_changed_condition_predictions.json	Q1 changed condition	f566713ea615aa11	before `q1b` run
q3_predictions.json	Q3	885da65e70ab624f	before `q3` run
q4_predictions.json	Q4	e8a1819ad0634ffe	before `q4` run
q5_predictions.json	Q5	f2623da6050e7398	before `q5` run
q6_predictions.json	Q6	f8769cc9ab43... (f8769cc9ab439048)	before `q6` run
q7_predictions.json	Q7	e875dbf789bfaf6a	before `q7` run

The lab (reproducible with `bash scripts/run_all.sh`, about 2 minutes).

Component	What it is	Address
Authoritative DNS	One BIND process with 22 *views*, one per lab server, so each "server" has its own data and behaviour	127.0.0.10 – .25, .30, .40 – .43

Component	What it is	Address
Caching resolver	Second BIND instance forwarding `cache.example`	127.0.0.53
Third-party provider	Out-of-scope "provider" zones (CDN, status page, shop, help desk …)	127.0.0.30
Mock WHOIS	Lab-owned text server, answers `whois -h 127.0.0.1 -p 4343 <domain>`	127.0.0.1:4343
Search engine	Python emulation of `site:`, `filetype:`, `intitle:`, `inurl:`, phrases and `-term` over a 42-document synthetic corpus (5 archive snapshots)	local
CT-style log	Synthetic certificate-transparency list (60% of true names + stale entries)	local
Tester identities	One source IP per workflow (127.0.0.101 …) so the **server-side query log** can verify every tool-side count and detect contact with out-of-scope servers	127.0.0.101 – .140

The server-side log is the independent evidence for query counts and for scope compliance ("did the tester ever contact a third-party server?").

Honesty notes that apply to the whole report. (1) The lab models real behaviour but is small, so results show direction and size of effects, not field statistics. (2) Several of my own scripts had bugs that the evidence exposed; they are reported where they occurred (Q1, Q2, Q3, Q6, Q7). (3) A few predictions were wrong; each is marked ✗.

Q1 – Evaluate and Engineer a Footprinting Strategy (8 marks)

1.1 Footprinting strategy design (passive and active techniques, classified)

#	Technique	Tool / operator	Class	Why this class	What it yields	Risk to the engagement
T1	WHOIS lookup	`whois -h 127.0.0.1 -p`	**Passive**	Talks to a registry, not to	Registrant, dates,	May be stale or privacy-

#	Technique	Tool / operator	Class	Why this class	What it yields	Risk to the engagement
		4343 <domain>		the target's servers	registrar, listed name servers	redacted; contains personal data
T2	Search-engine operators	site: filetype:pdf intitle:"index of" "domain" -site:domain	**Passive**	Reads third-party indexes	Documents, host names, e-mail/phone, exposed directories	Out-of-scope and unverified pages appear in results
T3	Archived pages	archive snapshots of site:domain	**Passive**	Reads a third-party archive	Historical hosts and name servers	Outdated: lists hosts that no longer exist
T4	DNS record queries	dig @ns <domain> A MX NS TXT SOA CAA, nslookup -type=MX	**Active**	Packets reach the target's authoritative server and appear in its log	Live records, SPF/DMARC, vendors, serial numbers	Leaves a trace; must stay inside scope
T5	Sub-domain brute force	wordlist of 63 labels, dig … A per label	**Active**	One query per guess to the target's DNS	Existing host names	Noisy: bursts exceed a detection threshold (Q6)
T6	Zone-transfer attempt	dig @ns <domain> AXFR	**Active (intrusive)**	Requests the whole zone; only done because the authorization sheet permits it for the four lab targets	Complete zone if misconfigured	Must be explicitly authorized

#	Technique	Tool / operator	Class	Why this class	What it yields	Risk to the engagement
T7	CT-style log (changed condition)	local log lookup	**Passive**	Third-party public log	Names with certificates, stale entries	Unverified until confirmed

Ordered strategy (final, after the changed condition in 1.7). S1 confirm written scope → S2 passive harvest (T1, T2, T3, T7) → S3 build a candidate list (wordlist ∪ words from public documents ∪ passive names, de-duplicated) → S4 scope-guard every candidate and every CNAME/NS target → S5 active verification, one A query per candidate, paced → S6 delegation detection (NS cut) and in-scope child-zone check → S7 label every item (verified / unverified / out-of-scope / personal data) and record contradictions → S8 report.

How items are kept apart (requirement: out-of-scope, personal data and unverified results must not be treated like verified in-scope findings).

Class	Rule	Treatment in the report
F1 In-scope, verified	Name or record confirmed by an authoritative answer	Counts toward the attack surface
F2 In-scope, unverified or outdated	From a document/archive, but the authoritative server does not confirm it	Listed as *lead* or *outdated*; **not** counted as attack surface
F3 Personal data	E-mail address, phone number, named person	Stored separately, masked in the report, never used as a technical finding
F4 Out-of-scope	Domain not owned by the authorized party (vendors, look-alikes)	Recorded as a boundary; **never probed**
F5 Unverified external claim	Anonymous page with no source (e.g. paste claiming a password dump)	Flagged; not used

1.2 Authorized test targets (four contrasting targets)

Target (lab-owned)	Designed to be	Name server	How it was built
fortress.example	**Well protected**	127.0.0.10	Minimal records, SPF `-all`, DMARC `p=reject`, CAA, transfers refused, WHOIS redacted
leaky.example	**Leaks information**	127.0.0.11	11 host names incl. internal 10.20.1.x addresses, 4 TXT records (SPF with vendor names, 3 verification tokens), DMARC `p=none` with a staff mailbox, WHOIS shows a person and an old name-server set, documents with internal host names, an open directory index and a staff list
misconf.example	**Misconfigured DNS**	127.0.0.12	`allow-transfer { any; }`; internal names, SRV records for AD/Kerberos, a TXT record with a firmware version
minimal.example	**Minimal public footprint**	127.0.0.19	Apex A, null MX, SPF `-all`, nothing else; WHOIS redacted; one parked archive page

1.3 Predictions before collection (locked in `q1_q2_predictions.json`)

For each target the prediction fixed: whether WHOIS shows the registrant and whether its name servers match DNS; the counts of A/MX/NS/TXT records; the zone-transfer result; which hosts the wordlist finds; how many results `site:` returns; and how many archive snapshots exist. Overall expectation: low leakage for *fortress*, high for *leaky* (documents add `jira` and `staging`, which the wordlist cannot find), a zone transfer exposing about 14 names for *misconf*, and almost nothing for *minimal*.

1.4 Collected-source records (real tool output) and prediction versus observation

Commands and raw output: `evidence/q1/raw_commands.txt` (307 commands: 4 `whois`, 303 `dig`/`nslookup`), plus `whois_<domain>.txt`, `axfr_<domain>.txt`, `nslookup_mx_<domain>.txt` and `q1_results.json`. Server-side check: the lab's query log shows **299** queries from the tester IP (fortress 71, leaky 83, misconf 74,

minimal 71); the 4 missing from the 303 tool-side `dig`/`nslookup` commands are the 4 AXFR attempts, which BIND logs in a different category. Tool-side and server-side counts therefore agree.

Example raw output (WHOIS of `leaky.example`): registrant *Anil Verma*, organisation *Leaky Retail Pvt Ltd*, **Updated Date 2019-02-02**, name servers *ns1/ns2.old-hosting.example*. Example raw output (AXFR of `misconf.example`) begins with the SOA, then lines such as `_ldap._tcp … SRV 0 100 389 dc01.misconf.example.`, `hr-portal … A 10.30.0.12`, `dc01 … A 10.30.0.5`.

Target	WHOIS	DNS records (A / MX / NS / TXT)	AXFR	Wordlist hosts	site: results	Archive snapshots
fortress	registrant redacted; NS matches DNS	1 / 1 / 1 / 1	refused (0 RR)	2 (`www`, `mail`)	2	1
leaky	registrant visible; **NS differs from DNS**	1 / 1 / 1 / **4**	refused	**7** (`www mail vpn dev git backup intranet`)	**6** (+2 PDFs, +1 open index)	2
misconf	registrant visible; NS matches	1 / 1 / 1 / 1	**allowed, 21 RR, 14 names**	3 (`www mail vpn`)	3 (+1 PDF)	0
minimal	redacted, status `clientHold`	1 / null MX / 1 / 1	refused	**0**	**0**	1 (parked page)

Prediction	Observed	Result
fortress: registrant hidden, NS matches, 1/1/1/1 records, AXFR refused, 2 hosts, 2 search results, 1 snapshot	exactly that	✓
leaky: WHOIS NS ≠ DNS NS; 4 TXT; AXFR refused; 7 hosts; 6 search results; 2 snapshots	exactly that	✓
leaky: documents add `jira` and `staging` (not in	both found in the annual-report PDF and confirmed by `dig` (A	✓

Prediction	Observed	Result
wordlist)	records exist)	
misconf: AXFR allowed, about 20 RR, about 14 names; wordlist finds 3	21 RR (incl. both SOA copies), 14 names; 3 hosts	✓
minimal: nothing beyond apex; 0 search results; 1 snapshot	exactly that	✓
External mentions of `leaky.example` (out-of-scope or unverified): 2	**1** (the paste page). The CRM-vendor page says "Leaky Retail" but never writes the domain, so the `"leaky.example" -site:leaky.example` operator cannot find it	✗ — a real limitation of operator searches

Classification of the collected items (requirement: do not treat everything alike).

Target	Items	F1 in-scope verified	F2 outdated / unverified in-scope	F3 personal data	F5 unverified external
fortress	6	6	0	0	0
leaky	33	25	1 (`old-vpn` from the 2019 archive; no A record today)	6 (3 staff e-mails + 2 phones + 1 more e-mail, masked)	1 (paste claiming a password dump, 2020, anonymous)
misconf	25	24	0	1 (company phone in a document)	0
minimal	4	4	0	0	0

1.5 Cases where one source contradicts another

#	Contradiction	Sources	Resolution
C1	WHOIS lists `ns1/ns2.old-hosting.example`; DNS says the only NS is `ns1.leaky.example` (WHOIS last updated 2019)	WHOIS vs DNS	DNS is authoritative → WHOIS NS is stale; flagged as an *inventory risk*

#	Contradiction	Sources	Resolution
			(possible forgotten delegation)
C2	2019 archive says remote access is at old-vpn.leaky.example; DNS has no such name; 2023 archive and DNS agree on vpn	archive vs DNS	Archive item F2 (outdated)
C3	A paste page claims a password for leaky.example; nothing else supports it	external page vs all others	F5; never used
C4	fortress WHOIS is redacted but the DNS SOA contact is hostmaster.fortress.example	WHOIS vs DNS	Not a conflict but a *difference in disclosure*; DNS still exposes a role mailbox

1.6 Source-reliability analysis (does any source mislead?)

Source	Observed behaviour	Reliability for current state	Misleading or outdated?
Authoritative DNS (dig)	Answers carry the aa flag; every host later checked was confirmed	**High**	Only when views/NS differ (Q2 D4, D5)
WHOIS	Correct about dates and privacy; NS list wrong for leaky	**Medium**	**Yes** – stale NS; personal data
Search operators	Found documents with internal host names and an open /backup/ index; also surfaced one unverified claim; missed a vendor page that did not spell the domain	**Medium**	Yes – unverified and out-of-scope hits
Archived pages	Showed old-vpn and old NS	**Low** for now, high	**Yes** – outdated

Source	Observed behaviour	Reliability for current state	Misleading or outdated?
		for history	
Wordlist brute force	7/7 of the hosts it can guess; blind to names not in the list (jira, staging, payroll-db-03)	High precision, limited recall	No

Conclusion: **no single source is sufficient**. DNS alone found 7 of leaky's 9 documented hosts; documents found jira and staging; the zone transfer found 11 more names than the wordlist on misconf; and WHOIS/archives needed DNS confirmation before use.

1.7 Changed condition – 50 sub-domains and several third-party services

Target: example-corp.example (127.0.0.21), 50 true names = 33 first-party hosts, 8 CNAMEs to third-party services, 4 delegated departments (own name servers), 5 stale/dangling CNAMEs. Third-party zones sit on 127.0.0.30 and count as **out of scope**.

Original strategy v1: 63-word wordlist, one A query per word, follow every CNAME target to "verify" it. **Modified strategy v2:** wordlist ∪ words taken from the organisation's public documents ∪ host names in those documents ∪ CT-style log names, de-duplicated; scope guard before any CNAME/NS follow-up (record, never probe); delegation detection (referral with NS) followed by one query to the in-scope child server.

Predictions (locked): v1 recall about 52% with at least 2 out-of-scope contacts; v2 recall 88–96%, 0 contacts, 250–400 queries.

Measure	v1 original	v2 modified	Prediction
True names found (of 50)	26	**47**	v1 26 ✓ / v2 44–48 ✓
Recall	52%	**94%**	✓
False positives	0	0	–

Measure	v1 original	v2 modified	Prediction
`dig` commands (tool side)	65	109	v2 predicted 250–400 ✗ (de-duplication cut candidates from 144 to 105)
Server-side queries (tester IP)	65	109	agrees with tool side
Out-of-scope contacts (server-side, third-party view)	**2** (`cdn`, `shop` CNAME targets)	**0**	✓
Third-party services recorded but not probed	–	11	–
Candidates rejected as non-existent	–	58	–

Category (true count)	v1 found	v2 found
First-party hosts (33)	24	32
Third-party CNAMEs (8)	2	8
Delegated departments (4)	0	**4**
Stale dangling CNAMEs (5)	0	3

Which source found what (v2). Wordlist 26 names (10 only it found); CT log 30 names (4 only it found); document words 12 names (4 only they found = the four department zones); document host names 13 (0 unique, all also found by others). **v2 misses 3 names** (`legacy-vpn`, `test`, `test-old`) that appear in no source – the recall ceiling of any OSINT-plus-wordlist method.

A bug found by the evidence. The first v2 run (`evidence/q1b_results_run1_buggy.json`) reported 86% and 0 delegated departments, because my delegation test searched for `" NS "` with spaces while `dig` separates fields with tabs. Fixing the pattern raised recall to 94% and found all 4 departments. The first run is kept as evidence.

Defence: is the original strategy suitable for large-scale authorized reconnaissance? No, not unchanged. It has low recall (52%) and, more seriously, it leaves scope: every CNAME followed is a request to somebody else's server. v2 keeps the same tools but changes the order (scope guard before follow-up), the sources (passive names first) and the candidate construction, and it reaches 94% recall with 0 scope violations at 109 queries. Two limits remain and are carried to Q6: unpaced active bursts exceed a plausible detection threshold, and 6% of names are in no source at all.

Evidence Expected checklist – Q1: strategy design ✓ (1.1) · authorized test targets ✓ (1.2) · collected-source records ✓ (1.4, raw files) · prediction-versus-observation ✓ (1.3–1.4, 1.7) · source-reliability analysis ✓ (1.6) · scope analysis ✓ (1.1 classes, 1.7 categories) · justified final strategy ✓ (1.1 steps, 1.7).

Q2 – Build a Reliable DNS Footprinting Workflow (7 marks)

2.1 The workflow (`scripts/q2_dns_workflow.py`; dig + nslookup)

Step	Action	Commands (example)	Interpretation rule
W0	Authorization sheet check: which servers and which actions are allowed (AXFR only where `axfr_authorized = true`)	–	AXFR is **not** attempted on a server without that flag (demonstrated on `cache.example`)
W1	SOA from **every** listed name server	`dig @ns dom SOA`	Different serials = out-of-sync servers (stale data)
W2	NS, A, MX, TXT at the apex, each against each name server	`dig @ns dom A …`	Compare answers across servers; record `aa` flag and rcode
W3	Targeted names: _dmarc, www, wordlist hits	`dig @ns _dmarc.dom TXT`	A name that exists only on one server is a *stale or split* signal
W4	MX and CNAME targets resolved	`dig @ns mail.dom A`	MX without address = broken/stale; CNAME to another domain = third-party, **not** followed

Step	Action	Commands (example)	Interpretation rule
W5	`nslookup -type=TXT dom ns` cross-check	`nslookup`	Same data through a second client
W6	AXFR where authorized, with and without TSIG	`dig @ns dom AXFR [-y hmac…]`	REFUSED = control present (a positive finding)
W7	Vantage comparison (external vs internal source) and resolver-cache trace	`dig -b <src> …`, `dig @127.0.0.53 …`	Differences are caused by split-horizon, delegation or caching, not by errors
W8	Label: *fresh / stale / conflicting / referral / cached / third-party*	–	Never report a value at face value without its label

2.2 Authorized test cases (eight contrasting cases)

Case	Lab server(s)	What it represents
D1 secured	`fortress.example` @127.0.0.10	Correctly secured server; transfers refused
D2 leaky	`leaky.example` @127.0.0.11	Records that leak vendors, mailbox, internal ranges
D3 open AXFR	`misconf.example` @127.0.0.12	Allows zone transfer to anyone
D4 stale	`stale.example` @127.0.0.13 and .14	Primary updated 2026-11-01, secondary still serial 2025-03-01
D5 split-horizon	`split.example` @127.0.0.15	Different answers for an external and an internal client (source 127.0.0.2)
D6 delegation	`deleg.example` @127.0.0.16, child `dept.deleg.example` @127.0.0.17 and lame @127.0.0.18	Parent returns a referral; one child server is lame
D7 caching	`cache.example` authoritative @127.0.0.23 behind resolver	Cached answer after the authoritative

Case	Lab server(s)	What it represents
	@127.0.0.53, TTL 6 s	data changes
D8 countermeasure	tsig.example @127.0.0.20 and hardened.example @127.0.0.24 (same data as D3)	Transfers restricted by TSIG

Total: **317 traced queries** (322 commands, raw output in evidence/q2/raw_commands.txt, structured trace in evidence/q2/query_trace.json).

2.3 Prediction versus observation (record analysis and query trace)

Case	Predicted (locked)	Observed (real dig)	Match
D1	A=1 MX=1 NS=1 TXT=1; AXFR refused	same; Transfer failed.	✓
D2	TXT=4 (SPF + 3 tokens); leaks vendor names, DMARC mailbox, internal 10.x	TXT=4: v=spf1 … include:_spf.mailprovider.example include:crm-vendor.example ~all, Google, Atlassian, MS tokens; DMARC rua=mailto:secops-lead@leaky.example; dev = 10.20.1.15; 7 wordlist hosts; AXFR refused	✓
D3	AXFR succeeds, about 20 RR, new names (payroll-db-03, legacy-sftp, scada-gw), SRV for AD, TXT with firmware	21 RR, 14 names including all three, _ldap._tcp and _kerberos._tcp SRV, scada-gw TXT "plant-1 gateway fw v2.1.4 (vulnerable branch)"	✓
D4	NS1 ≠ NS2; www .50 vs .40; serial 2026110101 vs 2025030101; oldhost only on NS2; MX mail vs mail-old (no address); promo dangling CNAME on both	all as predicted: serials 2026110101 / 2025030101; oldhost NXDOMAIN on .13 and 192.0.2.99 on .14; mail-old.stale.example has no address; promo CNAME campaign.cloudprov.example. on both (third party → recorded, not probed)	✓
D5	External: 192.0.2.60, names apex/www/mail; internal: 10.5.0.10 plus intranet, ad,	External saw 3 names (192.0.2.60/.61); internal saw **7** (apex, www, mail = 10.5.0.x, intranet .20, ad .30, finance-db .9 and _ldap._tcp SRV)	✓

Case	Predicted (locked)	Observed (real dig)	Match
	finance-db, SRV		
D6	Parent: referral without AA, NS + glue; child ns1: authoritative, www.dept = 192.0.2.71; child ns2: REFUSED	Parent NOERROR, aa=false, referral NS ns1/ns2.dept.deleg.example; child ns1 aa=true, 192.0.2.71; ns2 REFUSED	✓
D7	First: 192.0.2.80, TTL ≤ 6; after the change, the resolver still returns the old value; after TTL expiry the new one	First 192.0.2.80, TTL 6; after the change the resolver returned 192.0.2.80 with TTL 5 while the authoritative server already answered 192.0.2.81 (aa); after 7 s resolver returned 192.0.2.81	✓
D8	No key: transfer fails; with key: succeeds	tsig.example and hardened.example: 0 RR without key, 22 lines with key (21 RR + the TSIG record)	✓

All eight case predictions held. Two bugs of mine appeared and were fixed: the SOA serial was first parsed from the wrong field (the report above uses the corrected run), and the lab once lost its loopback aliases during development, which is why every collector now **aborts** if dig reports an unreachable server instead of recording an empty answer as "nothing found".

2.4 How split-horizon, delegation and caching change what an external tester sees

Mechanism	Evidence	Effect on the tester	Engineering consequence
Split-horizon (D5)	External client: 3 names, public addresses. Internal client: 7 names, private 10.5.x addresses and an LDAP SRV record	An external tester **under-estimates** the internal surface; the internal view reveals what an insider or a compromised laptop would see	State the vantage point in every finding; repeat from an internal vantage point only if authorized
Delegation (D6)	The parent zone holds only the referral; the data lives on the child servers; one child server is lame (REFUSED)	Querying only the parent shows nothing about dept; a lame server can create inconsistent resolution and is itself a finding	Follow referrals to in-scope child servers; record lame servers; never follow referrals to a provider's own infrastructure

Mechanism	Evidence	Effect on the tester	Engineering consequence
Caching (D7)	The resolver served .80 for the remaining 5 s after the zone changed to .81	Data from a resolver can be older than the zone by up to the TTL; aa=false marks it	Query authoritative servers for evidence; use the resolver only to learn what ordinary users see; note TTL in the record
Out-of-sync secondaries (D4)	Two authoritative servers disagree on 3 of 5 answers	Which server was asked changes the result	Always query every listed NS and compare SOA serials

2.5 Zone-transfer risk evaluation (D3): does a misconfigured transfer meaningfully change the attacker's knowledge?

Measure	Without AXFR (all queries + wordlist)	With AXFR	Change
Host names known	4 (www, mail, vpn, ns1)	15 incl. apex (14 sub-names)	×3.8 (11 of 14 sub-names not found by the wordlist)
Internal (10.x) addresses	0	8 (hr-portal, payroll-db-03, legacy-sftp, dc01, dc02, fileserver, build, scada-gw)	+8
Service records (SRV)	0	2 (LDAP and Kerberos → domain controller dc01)	+2
Software/version leakage	none	firmware v2.1.4 (vulnerable branch) on scada-gw	+1
Exposure score (name 1, internal address 2, SRV 3,	**4**	**54**	**×13.5**

Measure	Without AXFR (all queries + wordlist)	With AXFR	Change
version TXT 3, sensitive keyword in name 2)			

Judgement. Yes, it is meaningful: the attacker moves from "public web and mail" to a map of the internal network, the domain controller, an HR portal, a payroll database and an industrial gateway with a known-weak firmware – enough to choose targets, write convincing phishing/pretexting lures ("payroll-db-03 maintenance") and plan lateral movement. The limit is also real: a zone transfer **gives no access**; the 10.x addresses are useless to an attacker who cannot reach that network, and a zone that only lists names already in public certificate logs would add little. So the **risk is the information advantage, not direct compromise**, and it is highest when internal names, SRV records and version strings share the same zone as public names.

2.6 Countermeasure and redesigned analysis (restrict transfers + TSIG)

Change: `allow-transfer { none; }` as the server default, and for the one legitimate secondary `allow-transfer { key labkey; }` (HMAC-SHA256 TSIG). Applied to `hardened.example`, which has the same records as `misconf.example`.

Observation (same tester, same commands)	Before: open	After: restricted + TSIG
AXFR without key	21 RR	**0 RR** (Transfer failed)
AXFR with the TSIG key (authorized secondary)	–	22 lines (21 RR + TSIG) – works as intended
Names an unauthenticated tester learns	15	**4** (only what the wordlist finds)
Exposure score for an unauthenticated tester	54	**4** (back to the no-AXFR baseline)
Wordlist brute force	3 hosts	3 hosts (unchanged – the control does not hide existing names)

Redesigned analysis. W6 now treats `REFUSED` as a *positive control* and records it, tests the same command with and without the key, and adds a residual-risk line: TSIG key theft or reuse, brute-force and NSEC-style enumeration, and the need to split internal names into a separate internal view (as in D5). The workflow's logic changed from "find the zone" to "find the zone **and prove the control**".

Evidence Expected checklist – Q2: DNS workflow ✓ (2.1) · record analysis ✓ (2.3) · authorized test cases ✓ (2.2, 8 cases) · prediction-versus-observation ✓ (2.3) · zone-transfer risk evaluation ✓ (2.5) · countermeasure redesign ✓ (2.6).

Q3 – Create a Competitive-Intelligence Awareness Utility (7 marks)

No social-engineering attack is performed. All "organisations", staff, vendors, phone numbers and e-mail addresses are synthetic; the output is an awareness assessment of what public text would let an attacker do, plus what the staff should notice.

3.1 Workflow architecture (modular; `scripts/q3_ci_awareness.py`)

```
SOURCE REGISTRY (plug-ins)                  CORE PIPELINE   run_pipeline(sources, countermeasures)
+-------------------------+                          +-------------------------------------------------------------+
| job_ads          operator |       | 1 search   : one documented operator per source                    |
| annual_report    operator | --> | 2 extract  : regex extractors -> (category, value)                 |
| press_release    operator |       | 3 verify   : flag third-party site / unverified / outdated         |
| public_profiles  operator |       | 4 de-dup   : merge same item found in several sources              |
| + conference_slides (new)|       | 5 map      : category -> technique + indicator + weight            |
| + code_share (new)       |       | 6 score    : weight x reliability factor x corroboration           |
+-------------------------+        | 7 mitigate : COUNTERMEASURE REGISTRY (plug-ins) re-scores          |
                                   | 8 report   : items, techniques, indicators, risk level             |
COUNTERMEASURE REGISTRY                      +-------------------------------------------------------------+
| CM-1 callback verification (new) | CM-2 job-ad & slide sanitisation (new) |
```

Sources and countermeasures are entries in two lists; `run_pipeline()` never changes when one is added (shown in 3.7).

3.2 Documented steps and source operators (where each item came from)

Every item stores `source`, `url`, `date` and flags, so provenance is recorded. Operators run against the lab's synthetic corpus.

Source	Operator template	Step documented	Provenance recorded
Job advertisements	site:{domain} inurl:careers	list careers pages → extract tools, shifts, reporting line, contact mailbox	URL, date
Annual reports	site:{domain} filetype:pdf intitle:"annual report"	open each PDF text → extract offices, executives, vendors, events, helpdesk phone	URL, date
Press releases	site:{domain} inurl:press	extract events, partners, locations	URL, date
Public profiles	site:profiles-hub.example intitle:"{Organisation}"	extract names, roles, phones, e-mail, tools	URL, site, third-party-site flag
(new) Conference slides	site:{domain} filetype:pptx	extract speakers, architecture, vendors	URL
(new) Code-sharing pages	site:code-share.example inurl:{slug}	extract hostnames, mailboxes, config hints	URL, third-party-site flag

3.3 Item-to-technique mapping and indicator the target should notice

Item category (weight)	Example (synthetic)	Technique it could enable	Indicator a target should notice
E-mail address (2)	careers@orchid-analytics.example	**Phishing**	Mail from a look-alike domain or an unusual request to the exact mailbox format
Phone number (3)	helpdesk +91-120-5550-4000	**Vishing**	Caller quotes a real extension/name and pressures for urgent action; caller-ID may be spoofed
Person + role (3)	"Sunita Rao, Finance Controller"	**Pretexting**, phishing	Request that invokes a named manager and bypasses the normal approval path

Item category (weight)	Example (synthetic)	Technique it could enable	Indicator a target should notice
Tool/product (2)	Okta, SAP, Cisco AnyConnect	**Phishing**, pretexting	Login page or "reset" message naming the real product the staff use
Vendor/partner (3)	LogiPrime Freight, SecureBadge Systems	**Pretexting**, phishing	Payment-detail change or invoice from a vendor you did not initiate
Location/office (2)	"Tower B, 4th floor, Sector 62"	**Tailgating**, pretexting	Visitor claiming to be a contractor at the named building without a pre-booked pass
Shift pattern (2)	"Shift 06:00–14:00"	**Tailgating**	Person following staff through a door at a known shift change
Event/date (2)	"ERP go-live 1 Dec 2026", "month-end close on the 28th"	**Phishing**, pretexting	Message tied to a real upcoming event asking for credentials or approvals
Visitor/badge policy (2)	"visitors sign in at reception", "badge access"	**Tailgating**, pretexting	Someone offering a "badge replacement" story or avoiding sign-in

Scoring (fixed before any run). Item risk = category weight × factor. Factor: ×1.0 normal; **×1.5** for e-mail/phone found on an unverified third-party page (fake contact details in circulation are themselves a lure); ×0.3 for other unverified items; ×0.4 if older than 2 years; ×1.2 if corroborated by a second source. Level for one source: Low < 8, Medium 8–17, High ≥ 18; for one organisation: Low < 12, Medium 12–29, High ≥ 30.

3.4 Test organizations (rich, sparse, misleading)

Organisation	Type	Public documents built	Design intent
Orchid Analytics (orchid-analytics.example)	**Rich**	2 job ads, annual report, press release, 2 staff profiles on a third-party site (+ slides and a code-share page for the extension)	Named staff, roles, phone, e-mail, vendors, shift, office, badge policy,

Organisation	Type	Public documents built	Design intent
			upcoming events
Basalt Works (basalt-works. Example)	**Sparse**	1 careers page, one "about" page, annual report (+ harmless slides and readme)	Almost no actionable detail
Mirage Logistics (mirage-logistics.example)	**Misleading**	2 press releases with *different* CEOs and phone numbers (2022 vs 2026), 1 careers page, a profile on a third-party site claiming to be the "verified CSO" with a look-alike e-mail domain and a phone for "urgent vendor payment changes", a satire page (+ a 2022 slide deck)	Outdated, contradictory and impersonation content

3.5 Predictions versus observations (locked in q3_predictions. json)

Quantity	Predicted	Observed	Result
Orchid: job ads / annual report / profiles level	High / High / High	**High (35.2) / High (35.2) / High (20.2)**	✓
Orchid: press release level	Medium	**Low (7.6)** – only 3 items, mostly event/location	✗
Basalt: every source	Low	Low (job ads 2.0, annual report 2.0; no press page matched)	✓
Mirage: job ads / press	Medium / Medium	Medium (8.0) / Medium (8.4)	✓
Mirage: impersonation profile	High	**Low (7.5)** – only 2 items, though both flagged and ×1.5	✗ (score is absolute; the *alert* matters more than the level – see 3.6)

Quantity	Predicted	Observed	Result
Organisation level Orchid / Basalt / Mirage	High / Low / Medium	**High 68.2 / Low 4.0 / Medium 23.9**	✓
Item counts Orchid / Basalt / Mirage	35 / 2 / 11	28 / 2 / 10	✗ for Orchid and Mirage: I summed per-document ground truth and forgot that the same person, vendor or mailbox appears in several documents and is merged
Most dangerous source	public profiles	**annual report** (35.2 total, 22.0 unique risk); job ads equal in total (35.2, 16.0 unique); profiles have the highest *risk per item* (2.9 vs 2.7 vs 2.5)	half right

Extraction accuracy against planted ground truth (q3_extraction_eval.py): first extractor **53 of 60 planted items (88.3%)**; after fixing executives phrased "appoints X as CEO" / "X is now CEO" and de-duplicating people by name, **58 of 60 (96.7%)**. The remaining misses are the unnamed "verified CSO" (no name to extract) and one tool name; known precision issue: two overlapping patterns can label a person with a neighbour's role (the person is still counted once).

3.6 Does the public information meaningfully increase social-engineering risk? Which source is most dangerous?

Organisation	Items	Total risk	Level	Risk by technique (phishing / vishing / pretexting / tailgating)
Orchid (rich)	28	68.2	**High**	48.2 / 6.0 / 52.2 / 14.0
Mirage (misleading)	10	23.9	Medium	– (4 of 10 items carry flags)

Organisation	Items	Total risk	Level	Risk by technique (phishing / vishing / pretexting / tailgating)
Basalt (sparse)	2	4.0	**Low**	–

Yes, meaningfully – and by combination, not by any single item. Orchid's score is **17×** Basalt's. One document, the annual report, holds three named executives, three vendors, a go-live date, two locations, a badge/visitor rule and a helpdesk number; read together they supply the *who, whom to impersonate, why now and where*. Pretexting (52.2) and phishing (48.2) dominate; vishing is small only because just one phone number is published. **The most dangerous single source is the annual report by total and unique risk; the public profile is the most dangerous per item** because one page joins a name, a role, a direct phone, a mailbox and the tool that person administers.

Limit of the model: the score is additive, whereas real risk is combinatorial (a named approver plus a vendor plus a month-end date is worth more than their sum). The lab therefore under-states the effect of combinations.

Misleading information is a risk of its own. In Mirage's case the outdated CEO and the older phone are flagged (×0.4); the *impersonation profile* is flagged and its fake mailbox/phone are *raised* ×1.5, because a "verified" page that tells staff to phone a number for urgent payment changes is a ready-made vishing lure. Recommended action is an awareness notice and a take-down request, not just a score.

3.7 Extending without restructuring: two new sources and two new countermeasures

Change made: **2 entries appended to the source registry** (conference slides, code-sharing pages) and **2 functions appended to the countermeasure registry** (each 3–4 lines); `run_pipeline()`, the extractors and the scoring code were not edited.

Orchid scenario	Items	Risk	Change	Level
Base (4 sources)	28	68.2	–	High
+ 2 new sources (slides: 6 items, 15.2; code-share: 1 item, 2.0)	33	**78.8**	**+15.5%**	High
+ CM-1 callback verification only	33	63.8	−19.0% vs 78.8	High
+ CM-2 job-ad and slide sanitisation only	23	58.0	−26.4%	High

Orchid scenario	Items	Risk	Change	Level
+ both countermeasures	23	**43.0**	**−45.4%**	High (> 30)
Basalt before / after everything	2	4.0 / 4.0	none	Low
Mirage base / + sources / + countermeasures	10 / 11 / 8	23.9 / 25.3 / 12.4	+5.9% / −51%	Medium

- **CM-1** (callback verification procedure for payment, vendor and password-reset requests): halves the exploitability of phone, vendor and named-person items.
- **CM-2** (sanitise job ads and slides: no product names, shift times, exact floors or badge rules): removes those items from the public record — the larger single effect.
- Prediction check: new sources +20–30% ✗ **(+15.5%)**; countermeasures −35–45% ✓ (−45.4%, at the edge). The slides added a lot of risk (6 items) because a conference talk by the IT manager re-publishes his e-mail, tool names and vendor; the code-share page added little.
- Even with both countermeasures Orchid stays **High** (43.0): awareness controls reduce but do not remove exposure; the report recommends keeping the annual-report disclosures under review.

3.8 Recommended countermeasures (recorded output)

Exposure	Countermeasure	Owner	How to measure
Named approvers + vendors + payment dates	Out-of-band callback for any payment or bank-detail change (CM-1)	Finance	% of changes verified by callback
Tool names, shift times and floors in job ads	Pre-publication review template (CM-2)	HR and Security	items per advert in a quarterly re-scan
Direct phone and mailbox on third-party profiles	Role mailboxes; ask staff to remove personal phone	HR	staff profiles listing direct phone
Fake "verified" profile	Brand-abuse monitoring and take-down; official contact page	Communications	time to take-down

Exposure	Countermeasure	Owner	How to measure
Visitor/badge policy and shift pattern public	Visitor escort rule; anti-tailgating drill; badge-challenge training	Facilities	tailgating test pass rate (internal, consented)
Look-alike e-mail domain	DMARC p=reject; external-sender banner	IT	spoofed mail blocked

Files: `evidence/q3_results.json`, `evidence/q3_items_with_mapping.json` (every item with source, URL, flags, technique, indicator and risk), `evidence/q3_extraction_eval.json`. The first Q3 summary attributed a shared item only to the alphabetically first source, which hid the profile source's risk; it was replaced by non-exclusive attribution and the earlier file is kept as `q3_results_run1_exclusive_attribution.json`.

Evidence Expected checklist – Q3: workflow architecture ✓ (3.1) · source operators ✓ (3.2) · item-to-technique mapping ✓ (3.3) · test organizations ✓ (3.4) · prediction-versus-observation ✓ (3.5) · risk evaluation ✓ (3.6) · recommended countermeasures ✓ (3.7–3.8).

Q4 – Design a Reconnaissance Data-Validation Mechanism (6 marks)

4.1 Validation design (`scripts/q4_validate.py`)

Every collected item has `value`, `type`, `source`, `age_days`. Four checks, each with a measurable rule:

Check	Rule	Outcome if it fails	Why this class
Scope	Host equals `example-corp.example` or ends with `.example-corp.example` (label-boundary match) or IP in 198.51.100.0/24	**REJECT** (non-recoverable)	Re-querying cannot make an out-of-scope item valid
Duplication	Normalised key (lower-case, trailing dot removed) already seen	**DUP** (dropped, provenance merged)	Already represented
Freshness	Age ≤ limit per source: DNS 30 d, CT/search/archive/paste 90 d, WHOIS 365 d	**RECHECK** (recoverable: re-query confirms or	A fresh query can fix it, *if the target is in scope*

Check	Rule	Outcome if it fails	Why this class
		refreshes)	
Source reliability	DNS 0.95, CT 0.8, WHOIS 0.7, search 0.5, archive 0.3, paste 0.1; threshold 0.6; unverifiable below 0.2	$0.2 \leq r < 0.6$: **RECHECK** (confirm with a second source); $r < 0.2$: **REJECT** (unverifiable)	Low reliability is recoverable only if a better source exists

Recoverable versus rejected. *Recoverable (RECHECK)*: stale or lower-reliability data about an in-scope target → a re-query or second source can promote it to PASS. *Rejected (REJECT)*: out-of-scope (look-alike or foreign domain, IP outside the CIDR) or unverifiable (reliability < 0.2) → never re-queried. Verdict priority: REJECT > DUP > RECHECK > PASS. A failed freshness or reliability check **immediately dispatches a re-query** (eager re-check); this is realistic and is what makes check order matter for scope safety.

Effort measure = basic operations actually executed: scope 2 (domain and CIDR test), duplicate 1, freshness 1, reliability 1 + number of other reports of the same value (corroboration scan).

4.2 Test sequences (adversarial reconnaissance inputs), predictions and results

Specifications and predicted counts were locked before the run. Adversarial members include look-alike domains (`notexample-corp.example`, `example-corp.example.evil.example`, `example-corp.exampl3.example`, `xn--exmple-corp-8ib.example`, `www.example-corp.example.attacker.example`), case and trailing-dot duplicates, stale items on out-of-scope hosts, and a paste claim with reliability 0.1.

Sequence	Items	Composition	Predicted PASS/REJECT/DUP/RECHECK	Observed	Match
S1 all valid	20	20 valid	20 / 0 / 0 / 0	20 / 0 / 0 / 0	✓
S2 out of scope	20	10 valid, 6 foreign, 4 look-alike	10 / 10 / 0 / 0	10 / 10 / 0 / 0	✓

Sequence	Items	Composition	Predicted PASS/REJECT/DUP/RECHECK	Observed	Match
S3 stale	20	12 valid, 6 stale, 2 stale on out-of-scope hosts	12 / 2 / 0 / 6	12 / 2 / 0 / 6	✓
S4 duplicated	20	12 valid, 8 duplicates (exact, UPPER, trailing dot)	12 / 0 / 8 / 0	**11 / 0 / 9 / 0**	✗ – my generator drew two identical host names among the 12 "valid" items (11 distinct); the validator was right
S5 mixed adversarial	48	15 valid, 10 out-of-scope, 8 stale (3 out-of-scope), 8 duplicates (2 of out-of-scope items), 3 unverifiable, 4 low-reliability	15 / 18 / 6 / 9	15 / 18 / 6 / 9	✓
S6 boundary	12	age = limit (3), age = limit + 1 (3), reliability = threshold (3), reliability just below (3)	6 / 0 / 0 / 6	6 / 0 / 0 / 6	✓

Boundary notes: age exactly equal to the limit passes (≤) and one day over is RECHECK; reliability exactly 0.6 passes (≥). For S6 the WHOIS reliability is set to exactly 0.6 (the other sequences use 0.7) to land on the threshold. **Naive scope check:** replacing the label-boundary test by `endswith("example-corp.example")` lets `notexample-corp.example` through (**1 leak in S5**) – which is why the final rule requires a dot boundary.

4.3 Pass / fail / re-check analysis

Across S1–S6 (140 items) the redesigned validator produced PASS 74, REJECT 30, DUP 15, RECHECK 21, **0 bad items accepted**, **0 re-queries to out-of-scope hosts**. Re-check items are all in-scope and are the only ones that consume re-query effort (21 re-queries for 21 RECHECK items).

4.4 Does the order of the checks matter? (all 24 permutations over S1–S6)

Order (examples)	Total ops	Re-queries	**Out-of-scope re-queries**	Label mismatches	Bad items accepted
duplicate → scope → reliability → freshness (lowest ops)	**692**	21	0	3	0
scope → duplicate → reliability → freshness	699	21	**0**	**1**	0
scope → duplicate → freshness → reliability (chosen)	702	21	**0**	**1**	0
reliability → freshness → duplicate → scope (naive, scope last)	842	32	**5**	3	0
freshness → reliability → scope → duplicate (worst)	852	32	**5**	1	0

- **Effort:** ops range from 692 to 852 (a 23% spread) across the 24 orders.
- **Safety: 12 of the 24 orders** (every order that runs freshness or reliability before scope) send **5 re-queries to out-of-scope hosts** on these sequences – a scope breach caused only by ordering.
- **Accuracy of the final accept set:** identical for all orders (0 bad items accepted). **Label accuracy** differs: when duplicate runs before scope, out-of-scope duplicates are labelled DUP instead of REJECT (3 mismatches vs 1; the remaining 1 is the S4 collision).
- Prediction ("scope → duplicate → freshness → reliability is cheapest with zero breaches"): the chosen order is within **0.4%** of the lowest-ops order that is also label-correct (702 vs 699) and the lowest-ops order overall (692) mislabels scope rejects → ✓ in substance, though "reliability before freshness" is 3 ops cheaper.

4.5 Justified redesign of the order

Redesigned order: scope → duplicate → freshness → reliability (cheapest-and-most-decisive first, and no action that contacts a host before scope is established). Justification: scope is a constant-time test that terminally rejects the largest class; duplicate is a hash lookup; freshness is a comparison; reliability is the only check whose cost grows with the amount of data (corroboration scan), so it runs last and only on survivors. Trade-off: duplicate before freshness means a stale duplicate is dropped without refresh (acceptable: the first copy is validated). At 50 targets the two scope-first variants differ by 1 op (13,584 vs 13,583).

4.6 Scalability inquiry – 1 to 50 targets (59 items per target plus 4 shared third-party hosts)

Targets	Items	Ops naive (reliability first, scope last)	Ops original (scope → fresh → rel → dup)	**Ops redesign**	Out-of-scope re-queries naive / redesign	Redesign PASS / REJECT / DUP / RECHECK
1	59	350	303	**284**	2 / 0	30 / 16 / 5 / 8
5	295	1,928	1,605	**1,382**	10 / 0	147 / 76 / 44 / 28
10	590	4,558	3,410	**2,758**	20 / 0	295 / 151 / 91 / 53
20	1,180	12,134	7,672	**5,480**	40 / 0	579 / 301 / 200 / 100
50	2,950	52,614	25,288	**13,584**	100 / 0	1,397 / 751 / 558 / 244

Prediction	Observed	Result
Effort is linear in targets, $R^2 > 0.99$, for every order	redesign $R^2 = 0.99999$; original 0.989; **naive 0.967 (super-linear)** because the corroboration scan grows when many targets report the same shared host	✗ for naive and original
Redesign uses 25–45% fewer ops than naive	**74% fewer** than naive and 46% fewer than the original order at 50 targets	✗ (larger gain than predicted)

Prediction	Observed	Result
PASS share constant	47.4% (1,397 of 2,950) at 50 targets, 50.8% at one target	✓ (roughly)
DUP share rises with the number of targets	8.5% → 18.9%	✓

Analysis: the redesign's advantage **grows with volume** (1.2× fewer ops at one target, 3.9× at fifty) because it spends the expensive check only on the 47% that survive, and it never wastes re-queries on duplicates or out-of-scope items (244 re-queries instead of 615).

Evidence Expected checklist – Q4: validation design ✓ (4.1) · adversarial sequences ✓ (4.2) · pass/fail/re-check analysis ✓ (4.2–4.3) · ordering evaluation ✓ (4.4) · justified redesign ✓ (4.5–4.6).

Q5 – Evaluate Footprinting of a Hierarchical Organization (7 marks)

5.1 Hierarchical footprint representation

Each node has `name`, `kind` (domain, sub-domain, department, service), `scope` (in/out), `data` (public data present or not) and `children`. Levels: 0 organisation domain → 1 sub-domains → 2 departments / delegated zones → 3 services and third-party endpoints.

```
T3 irregular footprint (lab test structure)          example-corp.example (DNS-backed run)
root                                                 example-corp.example                (level 0)
+-- A -- A1                                          +-- 36 in-scope names               (level 1)
+-- B -- B1, B2 -- (B2_0, B2_1, B2_2), B3, B4, B5    |   +-- research / law / hostel / library
+-- C                (no public data, no children = EMPTY)  |   +-- www, lab each          (level 2: 8 nodes)
+-- D-cloud [OUT OF SCOPE] -- 9 more out-of-scope nodes  +-- 11 third-party CNAME services  (BOUNDARY: recorded, never probed)
+-- E -- E1 -- E2 -- E1   (cycle)
```

5.2 Traversal design with an explicit boundary condition

Breadth-first, level by level (`scripts/q5_hierarchy.py`):

```
frontier = [(root, 0)]
while frontier:
  level = []; next = []
```

```
for (node, depth) in frontier:
  if node in visited: count cycle/duplicate; continue # visited set
  if node.scope != 'in': record as BOUNDARY; never probe/expand; continue # explicit scope boundary
  if depth > MAX_DEPTH(10): continue # depth guard
  mark visited; 1 query; level.append(node)
  if node.data is False: record as EMPTY (children still expanded)
  next += children(node) # terminal node: no children
record(level); frontier = next
```

Boundary condition: *expand a node iff it is in scope, not visited and depth ≤ 10.* Properties: it terminates (finite nodes, visited set, depth guard); each in-scope node costs exactly one query; each level is recorded; out-of-scope nodes are noted once and their subtree is skipped.

5.3 Test structures, prediction versus observation

Structure	Description	Predicted visited / boundary / empty	Observed (levels)	Match
T1 shallow	root + 6 leaves	7 / 0 / 0	7 / 0 / 0 (1, 6)	✓
T2 deep	chain of 8 with a leaf on each	17 / 0 / 0	17 / 0 / 0 (1,1,2,2,2,2,2,2,2,1)	counts ✓; my predicted level shape (1,2,2,…) ✗
T3 irregular	A, B (5 children, one with 3), empty C, out-of-scope D of 10 nodes, cycle E	16 / 1 / 1; 9 skipped; 1 cycle	**16 / 1 / 1; 9 skipped; 1 cycle detection** (levels 1, 4, 7, 4)	✓
T4 balanced b=4, d=3	1 + 4 + 16 + 64	85 / 0 / 0	85 / 0 / 0 (1, 4, 16, 64)	✓
T5 empty	root, P (no data) → Q1 (no	5 / 0 / 3	5 / 0 / 3	✓

Structure	Description	Predicted visited / boundary / empty	Observed (levels)	Match
branches	data), Q2 (data), R (no data)			

5.4 How depth and branching affect effort and completeness (full tree sweep, 30 shapes)

Visited nodes equalled the formula $(b^{(d+1)} - 1)/(b - 1)$ in **all 30 cases**. Completeness is measured under a fixed budget of **100 queries**.

Branching b \ depth d	d = 3 nodes (budget coverage)	d = 4	d = 5	d = 6
1 (chain)	4 (100%)	5 (100%)	6 (100%)	7 (100%)
2	15 (100%)	31 (100%)	63 (100%)	127 (78.7%)
3	40 (100%)	121 (82.6%)*	364 (27.5%)*	1,093 (9.1%)
4	85 (100%)	341 (29.3%)	1,365 (7.3%)	5,461 (1.8%)
5	156 (64.1%)	781 (12.8%)*	3,906 (2.6%)*	19,531 (0.5%)

*(values marked * are 100/nodes; measured values for d = 6 and for b = 4 are quoted from q5_results.json.)* Findings: **depth costs linearly only for a chain; for b ≥ 2 effort grows exponentially with depth**, and branching multiplies the base of that exponent – at b = 4 a depth of 6 already means 5,461 queries. A fixed query budget therefore gives complete footprints for shallow or narrow trees and rapidly shrinking coverage for wide-and-deep ones, so large hierarchies need prioritisation by level (the traversal's level order does this naturally) or a budget cut-off that reports unfinished levels.

5.5 Empty branch and no-data nodes

T5 and branch C in T3 test this. The boundary condition treats "no public data" as **a recorded fact, not a stop signal**: P is counted and marked EMPTY, and Q2 beneath it is still reached (5 of 5 nodes visited, 3 empty). Had "empty" meant "stop", Q2 would have been missed (4 of 5). A node stops the traversal only if it is out of scope, already visited or a leaf.

5.6 Systematic traversal versus an ad-hoc approach (T3, same structure, 200 random runs)

Ad-hoc = pick any unexplored link at random, no visited set, no scope test, a budget of 32 steps (twice the systematic cost).

Measure	Systematic	Ad-hoc (mean of 200 runs)
Queries used	**16**	32 (budget exhausted)
In-scope nodes covered	100%	99.6% (189 of 200 runs complete)
Out-of-scope nodes visited	**0**	**9.96 per run; 200 of 200 runs entered the out-of-scope subtree**
Repeat visits	0	6.1
Termination	guaranteed	none: the E1↔E2 cycle repeats until the budget ends

Prediction check: ad-hoc visits out-of-scope nodes ✓, repeats ✓, loops ✓; "covers fewer in-scope nodes" ✗ – with twice the budget it covers almost everything. The honest conclusion is that systematic traversal wins on **scope safety, waste and termination**, not on raw coverage.

5.7 Real DNS-backed traversal (boundary tested against live lab servers)

`example-corp.example`, candidates = the 47 names found in Q1, tester 127.0.0.105, 56 `dig` commands.

Measure	Predicted	Observed
In-scope nodes visited	45	**45** (level 0: 1; level 1: 36; level 2: 8)
Boundary (third-party) services recorded	11	**11** (e.g. `cdn` → `edge.cloudprov.example`, `careers` → `jobs.hiringsite.example`, dangling `gone-beta.cloudprov.example`)
Department children found via delegation	8	**8**
Contacts with the third-party server (server-side log)	0	**0**

5.8 Boundary analysis and justified final design

Node situation	Test	Rule outcome	Verified by

Node situation	Test	Rule outcome	Verified by
Out-of-scope subtree	T3 D-cloud (10 nodes)	recorded once; 9 nodes never touched	boundary = 1, skipped = 9
Cycle	T3 E1 ↔ E2	second arrival counted as cycle, no second visit	cycles = 1
Empty node with children	T5 P	recorded EMPTY, children expanded	visited 5/5
Terminal node	leaves	no children, no further queries	all structures
Third-party CNAME	DNS-backed run	boundary; server log shows 0 contacts	11 boundary, 0 contacts
Depth guard (10)	not reached by any test (max depth 9)	would stop expansion	design check

Final design: breadth-first with a visited set, explicit scope boundary and depth guard. Reasons: the requirement to *record each level*, shallow (higher-value) nodes first, easy budget cut-off, and guaranteed termination. A depth-first variant uses less memory on very deep chains but cannot report levels as it goes and, with a budget, returns a narrow vertical slice instead of a broad one.

Evidence Expected checklist – Q5: hierarchical representation ✓ (5.1) · traversal design ✓ (5.2) · test structures ✓ (5.3) · prediction-versus-observation ✓ (5.3, 5.7) · boundary analysis ✓ (5.5–5.8) · justified final design ✓ (5.8).

Q6 – Evaluate Footprinting Approaches Under Constraints (7 marks)

6.1 The two approaches (plus one hybrid) and what each produces

Constraints fixed **before** the run (q6_predictions.json): stealth rate limit **20 DNS queries per minute** (assumed detection threshold), engagement window **30 minutes**, required coverage **90%**. Scope sizes: **10, 50, 200 and 1,000 hosts** (s10/s50/s200/s1000.example, same name server 127.0.0.22; 15% of the names in each zone are unguessable labels such as xwww-482).

Approach	Definition	Produces	Sends packets to the target?
A – Passive open-source	WHOIS + CT-style certificate log + search operators (3 lookups)	Names published in public sources (60% of true names in the lab) plus stale entries; **unverified**	No
B – Active DNS footprinting	Apex SOA/NS/MX/TXT, then a wordlist (63 labels) with numeric-suffix expansion (`www`, `www1`, `www2` … up to ceil(N/63) levels), one A query per candidate	**Verified** existing names the wordlist can guess	Yes – every query appears in the target's DNS log
C – Hybrid (recommendation candidate)	A first; then actively verify only the passive names, the stale extras and the 63 base words	Verified subset	Yes, fewer

6.2 The two workflows (applied to the same authorized target `sN.example`)

Workflow A (passive): (1) `whois -h 127.0.0.1 -p 4343 sN.example` → (2) read the CT-style log for the zone → (3) `site:sN.example` search → (4) label every name *unverified*; stale entries kept as leads. → 3 lookups, 0 packets to the target.

Workflow B (active): (1) `dig` SOA, NS, MX, TXT for the apex → (2) build candidates = 63 labels + 63 × K numeric variants with K = ceil(N/63) → (3) `dig @ns name A +short` for each → (4) a non-empty answer = *verified name*; empty = NXDOMAIN (wasted query). Run once **unthrottled** (to measure burst rate in the server log) and the stealth time **calculated** from the query count at 20 queries/minute.

6.3 Quantitative measures (real runs; tester IPs 127.0.0.120–.130; server log used for rates)

Scope N	**A passive:** lookups / coverage / stale false leads	**B active:** queries / coverage / wasted (NXDOMAIN)	B unthrottled: seconds / peak queries per 60 s	B time at 20 q/min	B feasible (≤ 30 min)?	**C hybrid:** queries / coverage / time
10	3 / 60% / 1	130 / 90% / 92.1%	2.05 s / **130**	6.5 min	yes	69 / 100% / ≈3.5 min
50	3 / 60% / 2	130 / 86% / 65.1%	2.18 s / **130**	6.5 min	yes	75 / 98% / ≈3.8 min
200	3 / 60% / 10	319 / 85% / 45.7%	5.29 s / **319**	15.9 min	yes	165 / 69.5% / ≈8.2 min
1,000	3 / 60% / 50	1,075 / 85% / 20.6%	17.34 s / **1,075**	**53.8 min**	**no**	684 / 62.3% / **34.2 min (no)**

Budgeted active run at 1,000 hosts. What the stealth rule actually allows is 20 × 30 = 600 queries (4 apex + 596 candidates): the run completed in exactly 600 queries and reached only **50.0% coverage – below passive-only (60%)**.

Every unthrottled active run exceeds the 20-queries-per-minute detection threshold (130 to 1,075 queries in the busiest 60 s) → **detected if not throttled** at all four sizes. The seconds column is supporting evidence only; conclusions use coverage, scope and detectability.

6.4 Predictions versus observations

Item	Predicted	Observed	Result
Passive coverage and lookups	60%, 3	60%, 3	✓
Passive stale leads	1 / 2 / 10 / 50	1 / 2 / 10 / 50	✓
Active coverage	90 / 86 / 85 / 85%	90 / 86 / 85 / 85%	✓
Active queries	128 / 128 / 314 / 1,120	130 / 130 / 319 / 1,075	✗ small: I used a 62-

Item	Predicted	Observed	Result
			word list; the real list has 63 labels
Time at stealth rate	6.4 / 6.4 / 15.7 / 56 min	6.5 / 6.5 / 15.9 / 53.8 min	≈ ✓
Impractical size	1,000	1,000	✓
Unthrottled bursts exceed 20 q/min	all sizes	all sizes	✓
Hybrid at 1,000: coverage / queries	70% / 716	**62.3% / 684**	✗
"The faster or the quieter approach is always better"	no	no (6.5)	✓

My first Q6 run was **invalid**: the lab's DNS servers had silently stopped, `dig` printed a connection error, and my script counted that text as "found". Coverage appeared as 90% everywhere. The collectors now abort when `dig` reports an unreachable server, and the numbers above come from the corrected, repeated runs. The per-minute peak was also first contaminated by earlier runs sharing a source IP; each run now has its own IP.

6.5 Redundant or repeated collection – where effort is wasted

Approach	Waste measured	Reading
B active	**92.1% / 65.1% / 45.7% / 20.6%** of queries returned NXDOMAIN at N = 10 / 50 / 200 / 1,000	A fixed 126-candidate list is far too big for 10 hosts and efficient only when the zone is large; the waste is a *sizing* problem
B active	One query type per candidate, so no repeated type collection (contrast with Q7's original workflow, which asks three types per candidate and wastes two thirds of its queries)	–

Approach	Waste measured	Reading
A passive	1–5% false leads (50 of 1,050 entries at N = 1,000)	Cheap to discard, but each needs a DNS check before use
C hybrid	84.6 / 31.0 / 13.7 / 8.4% NXDOMAIN	Verifying only known names removes most waste, at the price of missing unknown names
A and B together (Q1)	The same host (e.g. `www`, `vpn` on `leaky.example`) is found by DNS, documents and archives	Cross-source duplicates must be merged (Q4 duplication check); the lab's scale corpus has only one name list per zone, so cross-source duplication is **not** measured in Q6

6.6 Is the faster or the quieter approach always the better choice? No

Criterion	A passive	B active	Winner
Quiet (packets to target)	0	up to 1,075, bursts exceed threshold	A
Fast	milliseconds	2–17 s unthrottled, 6.5–54 min throttled	A
Complete (coverage)	60% at every scope; cannot reach 90%	85–90% at every scope	B
Reliable / verified	unverified, stale entries	verified by authoritative answers	B
Feasible under stealth + window	always	not at 1,000 hosts	A
Meets the 90% coverage requirement	never	only at 10 hosts (90%)	neither at large scope

The fastest and quietest approach is also the least complete; the most complete one is the loudest. **The better engineering choice depends on scope and on which constraint is binding.**

6.7 Defended recommendations

Context	Recommendation	Evidence
Small-scope engagement (about 10–50 hosts)	**Active B (after a free passive pass, i.e. A then B)**	86–90% coverage, verified results, 6.5 min at the stealth rate, no feasibility problem; passive alone leaves 40% undiscovered
Large-scope engagement (about 1,000 hosts)	**Passive first, then verify a prioritised subset** within the 600-query budget; ask the client for a higher rate or a longer window if ≥ 90% coverage is required	A gives 60% for 3 lookups; budgeted active gives 50% and full active would need 53.8 minutes; hybrid C needs 34.2 minutes, still over the window
Teaching	**B on small zones, with A run on the same target**	Students see raw dig output, the prediction-versus-observation gap and the NXDOMAIN waste; the zones are cheap and safe
Repeated real assessments	**A kept as a baseline and diffed each cycle; B only on new or changed names**	Avoids repeated collection (Q6.5) and keeps the target's log quiet

Limits: the 20 queries/minute threshold and 30-minute window are assumptions chosen for the experiment; stealth time is calculated, not measured; passive data is synthetic.

Evidence Expected checklist – Q6: approach definitions ✓ (6.1) · two workflows ✓ (6.2) · quantitative measures ✓ (6.3, 4 scope sizes) · redundancy analysis ✓ (6.5) · defended recommendation ✓ (6.7).

Q7 – Integrated Review: Diagnose, Redesign and Defend (8 marks)

Workflow selected: the active DNS footprinting workflow **W1** used as the original strategy in Q1 and as Approach B in Q6: a fixed candidate list (63 labels + one numeric suffix), **three queries per candidate (A, AAAA, TXT)**, report any answer, and **follow every CNAME target** to verify it.

A. Diagnose the limitation (2 marks)

Bottleneck: W1 reports false names on wildcard zones, spends most of its queries on information it does not use, leaves the authorized scope through CNAME follow-up, and its recall collapses when the zone is bigger than its fixed list. Evidence from earlier questions and a new test:

Evidence	Value	Source
Recall on the 50-name target	52% (26 of 50)	Q1.7
Contacts with out-of-scope third-party servers	2 (wordlist-only run); **6** in the Q7 run	Q1.7 / Q7 server log
Queries needed for 1,000 hosts	1,075 = 53.8 min at the stealth rate → infeasible	Q6
Queries answered NXDOMAIN at 10 hosts	92.1%	Q6
Wildcard zone (wild.example, 4 real names)	**126 names reported, 122 of them false (precision 3.2%)**	Q7

Candidate cause	Test	Verdict
Redundant collection	3 record types per candidate although only the A answer decides existence	**Yes** – 2 of every 3 queries are unnecessary
Unreliable source / answer	A wildcard answers every label → 126 "hosts"	**Yes** – no wildcard test
Risk of going out of scope	CNAME targets are queried on the third-party server; because the CNAME is returned in all three query types, each is followed **three times** (2 CNAMEs × 3 = 6 contacts)	**Yes** – redundancy amplifies the scope breach
Poor ordering	Follow-up happens *before* any scope test; the candidate list is fixed regardless of zone size	**Yes**

Candidate cause	Test	Verdict
	(recall 54.7% at 200 hosts)	

Underlying cause (primary): the workflow has no scope guard in front of the follow-up step and no check that an answer is genuine; **secondary:** redundant per-candidate queries and a fixed list.

B. Evaluation criteria (1 mark)

Locked in `q7_predictions.json` before running:

Criterion	Definition	Target
Coverage	true positives / true names	not lower than W1 on any zone
Precision (source reliability)	true positives / reported names; a name counts as reliable only if the authoritative answer differs from the wildcard answer	≥ 0.98
Number of queries	`dig` commands (tool side, cross-checked with the server log)	≤ W1 on every zone
Scope compliance	queries received by the third-party view from the tester (server side)	**0**
Time	seconds, supporting only	informational

C. Proposed and implemented redesign W2 (2 marks)

Design logic before implementation (ordered steps):

1. **R1 – scope guard first.** A CNAME/NS target outside the authorized domain is *recorded*, never queried.
2. **R2 – wildcard probe.** Query two random labels; if both return the same non-empty answer, remember it as the wildcard answer and discard any candidate whose answer equals it.
3. **R3 – one A query per candidate.** Other types only for confirmed names (a later phase).
4. **R4 – adaptive suffix expansion.** Continue to level k+1 only if level k produced a new name; stop at the first empty level.
5. **R5 – de-duplicate candidates.**

```
probe two random labels -> wildcard = answer if both equal and non-empty
level = 0
repeat:
  new = 0
  for c in candidates(level) (de-duplicated):
    a = query_A(c)
    if a is empty or (wildcard and a == wildcard): continue
    report c; new += 1
    for each CNAME target t in a: if not in_scope(t): record(t)  # never query
  if level >= 1 and new == 0: stop
  level += 1
```

Trade-offs introduced: (a) the stop rule has patience 1, so a *gap* level (no names at k, names at k+1) would end the search early; (b) a real host that happens to share the wildcard address is discarded; (c) W2 returns fewer record types per name (AAAA/TXT are collected later, only for confirmed names); (d) it costs 2 extra probe queries and one empty confirming level.

D. Validation and comparison under equivalent conditions (2 marks)

Both workflows ran against the same four zones, with the same label list and suffix function, from separate tester IPs, with the server log counting contacts. Zones: `wild.example` (wildcard), `example-corp.example` (50 names + third-party CNAMEs), `s50.example`, `s200.example`.

Zone (true names)	W1: reported / precision / recall / queries / out-of-scope contacts	W2: reported / precision / recall / queries / out-of-scope contacts
wild (4)	126 / **0.032** / 1.000 / 382 / 0	**4** / **1.000** / 1.000 / **195** / 0 (wildcard detected)
example-corp (51)	28 / 1.000 / 0.549 / 388 / **6**	29 / 1.000 / **0.569** / **258** / **0** (2 CNAME targets recorded, not queried)
s50 (51)	44 / 1.000 / 0.863 / 382 / 0	44 / 1.000 / 0.863 / **195** / 0
s200 (201)	110 / 1.000 / **0.547** / 382 / 0	171 / 1.000 / **0.851** / **321** / 0
Total	308 reported; **122 false**; **1,534 queries**; **6 out-of-scope contacts**	248 reported; **0 false**; **969 queries (−37%)**; **0 contacts**

The tool-side and server-side query counts were identical for every run. True positives rose from 186 to 248.

Prediction (locked)	Observed	Result
W1 reports all 126 candidates on the wildcard zone; W2 reports 3	W1 126; W2 **4**	partly ✗: the zone's own `ns1` is a real record that I had left out of the truth set (truth is 4, not 3); the first run scored it as a false positive. Truth sets were corrected and all runs repeated (first run kept in `q7_results_run1_truth_without_ns1.json`)
W1 out-of-scope contacts on example-corp: 2	**6**	✗ (each CNAME is returned by all three queries and followed each time)
W2 queries: wild 132, example-corp 258, s50 132, s200 321	195, **258**, 195, **321**	✓ for two; ✗ for wild and s50 because the real `ns1` made level 1 productive and added a level
W2 recall ≥ W1 everywhere	equal or higher on all four	✓
W2 recall on s200 about 85% vs W1 about 54%	85.1% vs 54.7%	✓

Does the redesign improve the selected criteria? Yes, with numbers: precision on the wildcard zone 3.2% → 100%; scope contacts 6 → 0; queries 1,534 → 969; coverage non-inferior everywhere and +30 points on the larger zone.

E. Defend the final engineering judgment (1 mark)

Objective met: the diagnosed limitations – false names, scope leakage, redundant queries and size-blind coverage – are each removed or reduced with evidence, and no criterion got worse on any of the four test zones.

Where the original may still be preferable (stated honestly; only the first is supported by the data):

Case	Reason	Evidence status
A single known-clean name or tiny zone where AAAA and TXT are wanted in the same pass	W1 returns three record types per hit; W2 needs a second phase	Design difference, not a measured result

Case	Reason	Evidence status
A zone whose host numbering has a gap	W2's "stop at first empty level" could stop before the later names; W1's fixed list would be unaffected up to its limit	**Untested hypothesis** – no gap zone was built
The client explicitly authorizes probing the third-party endpoints	W1's CNAME follow-up would be allowed	Policy case; not tested

Final decision: adopt **W2** as the footprinting workflow, keep W1's multi-type collection as an *optional second phase* on confirmed names, and add a gap-tolerant stop rule (patience 2) as the next experiment. W2 also removes the Q6 weakness (queries not tied to scope size), but at 1,000 hosts it still needs the passive-first strategy of Q6 to stay inside the stealth budget.

Limits of the evidence: deterministic lab, one run per workflow per zone (no variance estimate), zones of 4 to 201 names, synthetic wildcard behaviour, and my own ground-truth error (`ns1`) that was found and corrected.

Evidence Expected checklist – Q7: A diagnosis with evidence and cause ✓ · B measurable criteria (coverage, queries, reliability, scope) ✓ · C redesign with ordered steps and trade-offs ✓ · D quantitative comparison under equal conditions ✓ · E judgment with cases where the original is preferable ✓.

Submission requirements – where each item is

Required item	Where
Assignment report with evidence for Q1–Q7	this document
Scripts / documented command sequences for all implemented workflows	`scripts/` in the package (`q1_footprint.py` … `q7_redesign.py`, `run_all.sh`) and the raw dig/nslookup/whois command lists in `evidence/*/raw_commands.txt`
Student-designed authorized test targets and adversarial cases	`lab/zones/*.zone`, `lab/named_auth.conf`, `lab/data/corpus.jsonl`, Q4 sequences S1–S6, Q5 structures T1–T5, Q7 wildcard zone

Required item	Where
Raw collected data and tool outputs	evidence/ (raw commands, AXFR dumps, WHOIS replies, JSON results, q2/query_trace.json) and lab/var/query.log (server-side log)
Prediction-versus-observation comparisons	tables in each question; prediction files with hashes in predictions/
Tables and graphs	tables throughout; results/all_charts.png (eight panels)
Source, query, item and scope counts	Q1 1.4/1.7, Q2 2.2, Q3 3.7, Q4 4.6, Q5 5.3–5.7, Q6 6.3, Q7 D
Design representation / pseudocode for redesigned workflows	Q4 4.5, Q5 5.2, Q7 C

Viva quick-reference (numbers to remember). Q1: recall 52% → 94%, out-of-scope contacts 2 → 0, 109 queries. Q2: AXFR 21 RR / 14 names, exposure score 4 → 54, TSIG brings the unauthenticated tester back to 4. Q3: Orchid 68.2 (High), Basalt 4.0 (Low), both countermeasures −45.4%. Q4: scope first; 12 of 24 orders cause out-of-scope re-queries; 74% fewer operations than the naive order at 50 targets. Q5: BFS with visited set and scope boundary; 16/1/1 on the irregular tree; ad-hoc entered out-of-scope nodes in 200 of 200 runs. Q6: active 130 → 1,075 queries; 53.8 minutes at the stealth rate; budgeted active 50% < passive 60%. Q7: wildcard precision 3.2% → 100%, 1,534 → 969 queries, 6 → 0 scope contacts.

Mistakes found and fixed while doing this work (all kept as evidence): Q1 delegation test used the wrong separator (recall 86% → 94%); Q2 SOA serial parsed from the wrong field; Q3 source attribution depended on alphabetical order and the extractor missed "appoints X as CEO"; Q4 my generator created an accidental duplicate; Q6 first run counted connection errors as findings and shared a source IP between runs; Q7 ground truth omitted ns1 and the first log counts were doubled by reusing source IPs. The final numbers come from one clean run_all.sh execution after the lab was restarted.